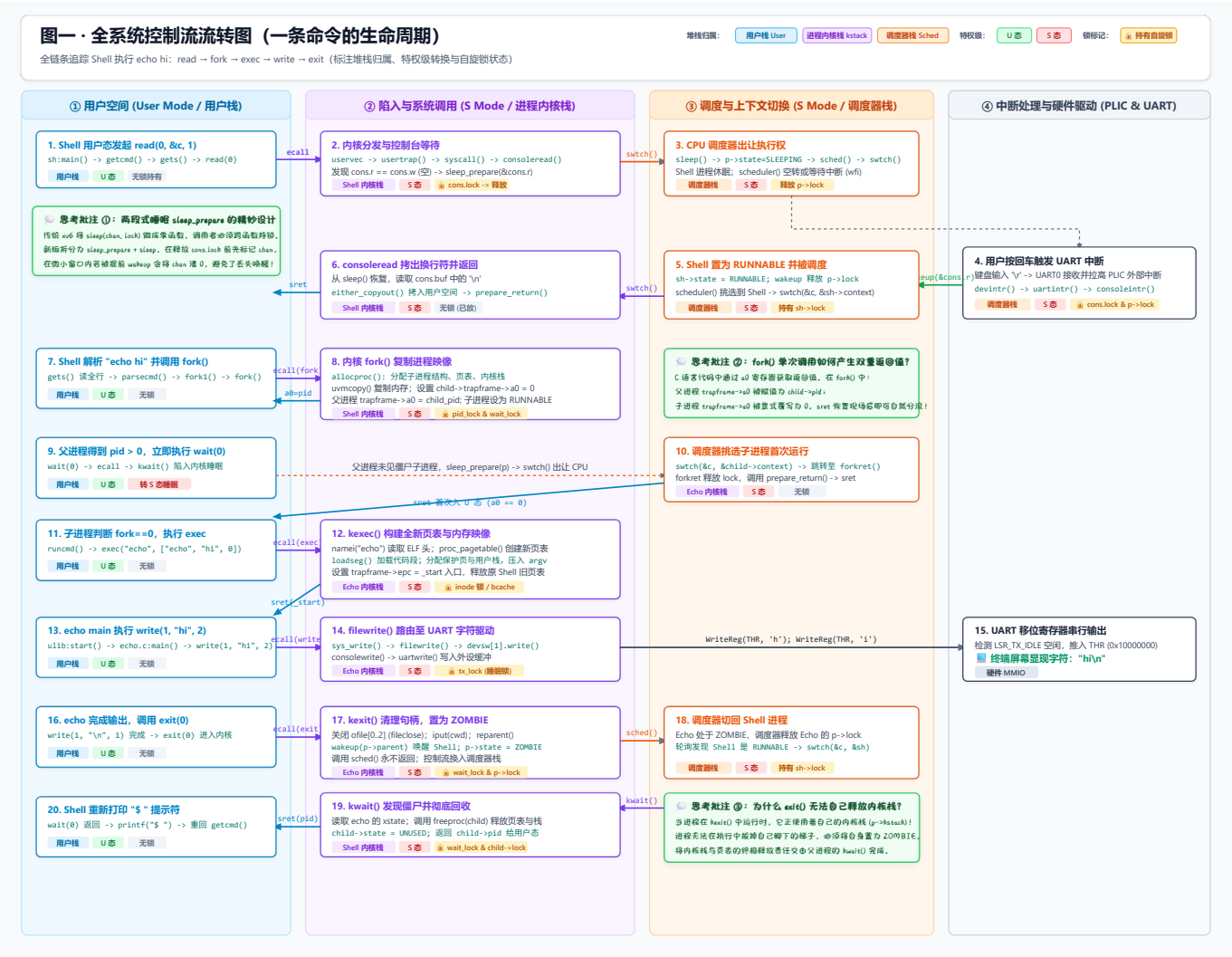


实验 0（热身）

第一章 材料一：全系统控制流流转图（模块级）



第二章 材料二：核心数据结构全景快照

图二 · 核心数据结构状态映射图

关键截图时刻: exec("echo") 刚执行完毕, echo 首条用户指令尚未运行 (进程表, SV39 多级页表映射树及标准输出文件表全景)

当前特征: p->sz = 0x3000 | epc = 0x0 (_start) | sp = 0x2feb | a0 = 2

进程表快照 (struct proc proc[NPROC])

```
proc[0] initproc (PID: 1, name: "init")
state: SLEEPING (睡眠在自身 &proc[0], 等待孤儿退出)
parent: 0 (根进程) | sz: 4096 (1页代码)
pagetable: init_pagetable (独立三级页表)
ofile[16]: [0:cons, 1:cons, 2:cons, 3..15: 0]
```

```
proc[1] sh (PID: 2, name: "sh")
state: SLEEPING (在 kwait() 中 sleep, prepare(p) 休眠)
parent: &proc[0] (init) | sz: 16384 (4页内存)
chan: &proc[1] (等待子进程 exit 唤醒)
pagetable: sh_pagetable
ofile[16]: [0:cons, 1:cons, 2:cons, 3..15: 0]
```

```
proc[2] * echo (PID: 3, name: "echo") - [RUNNING]
state: RUNNING (正处于 S 态, prepare, return 未尾)
parent: &proc[1] (指向父进程结构体)
sz: 12288 (0x3000) (3页: 代码段+保护页+栈)
pagetable: echo_pagetable (→ 右侧多级映射树)
kstack: 0x3ffffee000 (独占 4KB 内核栈)

p->trapframe (栈占用情况, 物理地址已映射):
epc : 0x0000000000000000 (ELF 入口点 _start)
sp : 0x00000000000002feb (对齐后的用户栈顶)
a0 : 2 (argc) | a1: 0x2feb (argv 指针数组地址)

ofile[16]: [0:cons, 1:cons, 2:cons, 3..15: 0]
```

```
proc[3 ... 63] (剩余 61 个空闲槽位)
state: UNUSED | pid: 0 | pagetable: 0 | trapframe: 0
```

❖ 思考批注 ①: 标准输出文件描述符继承与引用计数哲学

Shell 在启动时打开 console 3 次 (fd 0,1,2), 对应 file 结构的 ref+3; 在 fork() 时, 通过 filedup() 将父进程全部有效 ofile 的 ref 递增; 当 echo 进程诞生时, 系统全局控制 file 结构的 ref 已达到 9! 这保证了 echo 执行完毕 exit 时执行 fileclose() 仅递减计数, 绝不会因子进程控制位后逻辑, 体现了 Unix 优雅的引用计数管理。

❖ 思考批注 ②: exec() 释放旧页表为何不会发生逻辑?

在 exec() 中, 内核首先分配全新的独立页表 pagetable 并加载新代码; 待全部数据拷贝成功且无可逆错误后, 才将 p->pagetable 原子切换; 最后调用 proc_freepagetable(oldpagetable, oldsz) 释放旧页表物理内存; 这一机制保证了即便加载失败, 原进程状态依然完整可控。

Echo 进程虚拟地址空间与 SV39 三级页表映射

TRAMPOLINE (共享基址代码页, 4KB)

VA: 0x3fffffe000 - 0x3fffffefff (MVA - 4KB)

物理页: 内核代码 trampoline
PTE: V=1, R=1, W=0, X=1, U=0

TRAPFRAME (进程独占用户现场页, 4KB)

VA: 0x3fffffe000 - 0x3fffffefff (TRAMPOLINE - 4KB)

物理页: p->trapframe (独立分配)
PTE: V=1, R=1, W=1, X=0, U=0未映射巨型地址空洞 (Unmapped Virtual Gap)
0x0000003000 - 0x3fffffe000 (约 256GB 空间, 访问即触发缺页异常)

用户栈 (User Stack, 1页 4KB, 向下生长)

VA: 0x0000002000 - 0x0000002fff (p->sz = 0x3000)
栈顶内容: argv[0]="echo", argv[1]="hi", argv 指针数组, sp=0x2feb0物理页: 用户栈物理页 (kalloc)
PTE: V=1, R=1, W=1, X=0, U=1

栈保护页 (Guard Page, 1页 4KB) - [无 PTE_U 权限]

VA: 0x0000001000 - 0x0000001fff

物理页: 占用物理页 (有映射)
PTE: V=1, R=1, W=0, X=0, U=0

代码段与数据段 (ELF Text & Data Segment, 1页 4KB)

VA: 0x0000000000 - 0x0000000fff

物理页: ELF 内容页 (从磁盘加载)
PTE: V=1, R=1, W=1, X=1, U=1

SV39 三级页表级联转换结构 (Hardware MMU Walk)



RISC-V 39 位虚拟地址结构 (SV39 Virtual Address Breakdown):

VPN[2] (9b): 30..38 VPN[1] (9b): 29..21 VPN[0] (9b): 20..12 Offset (12b): 11..0

64 位页表项结构 (Page Table Entry Bits):

PPN [53:10] (44 位物理页号) RSM D A G U X W R V

V=有效位 | R/W/X=读写执行 | U=用户态可访问位 | A/D=访问/脏位

❖ 思考批注 ③: Guard Page 的“有页无权”与双页对称映射的深刻意义

- 栈保护页 Guard Page 为什么需要 PTE.U=1 仅读/无写? 若直接置 PTE.U=0 (无映射), 后续 lab3/lab5 实现栈帧分配或写时复制, 缺页异常会被判定为“正常的堆栈分配失败”并为其分配物理页, 从而失去保护! 保留有效位能精确触发权限违规!
- TRAMPOLINE 为什么必须在用户页表与内核页表完全映射到同一个最高页? 在写 satp 切换页表的最后一刹那, PC 依然在当前物理页连续取指, 若两个虚拟地址空间映射不同, 下一串指令立即会发生取指失败 (Instruction Page Fault, cause=12) 导致直接死锁崩溃!

文件表快照与标准输出引用链路

echo->ofile[NOFILE] (文件描述符表)

```
ofile[0] (stdin) -> ftable.file[1] (console)
ofile[1] (stdout) -> ftable.file[1] (console) *
ofile[2] (stderr) -> ftable.file[1] (console)
```

系统全局文件表 (struct file ftable.file[])

type : FD_DEVICE (字符设备文件)

readable : 1 (可读)

writable : 1 (可写)

major : 1 (CONSOLE) (设备驱动主编号)

ref : 9 (init/sh/echo 各持有 3 个 fd 共享)

ip : 0 (设备文件不占用磁盘 inode)

根据 major=1 索引

全局设备驱动开关表 (struct devsw devsw[])

read : console_read() (读控制台缓冲)

write : console_write() (写控制台外设) *

分发机制: filewrite() 检测到 FD_DEVICE 即调用 devsw[major].write

UART 驱动输出实现 (uartc: uartwrite)

- acquiresleep(&tx_lock); 获取发送锁
- 循环检查 LSR 寄存器状态: ReadReg(LSR) & LSR_TX_IDLE
- 成立则推入寄存器: WriteReg(THR, 'h');
- releasesleep(&tx_lock); 释放发送锁

硬件 MMIO 内存映射寄存器 (16550A UART)

UART0 基地址: 0x10000000

THR (Transmit Holding): 0x10000000 (偏移 0)

LSR (Line Status): 0x10000005 (偏移 5)

向 物理总线发射串行信号, 屏幕打出 "hi"

❖ 思考批注 ④: 设备抽象为何不暴露内核 inodes?

在 Unix-like 设计中, FD_INODE 针对磁盘普通文件设置目录, 必须依赖磁盘 inode 寻址逻辑; 而字符设备 (如控制台) 被抽象为直接与硬件页对齐。因此 file 结构中仅记录主设备号 major=1, 将文件读写直接映射到设备寄存器表 devsw, 体现了“一切皆文件”但机制完全解耦的经典哲学。

第三章 材料三：一次时钟中断的微观旅程

